LAB # 01

Submitted By: Muhammad Ahmad Malik

Student ID: 41171

Submitted To: Mam Zilla Huma

Subject Name: Advance Data Base Management (LAB)

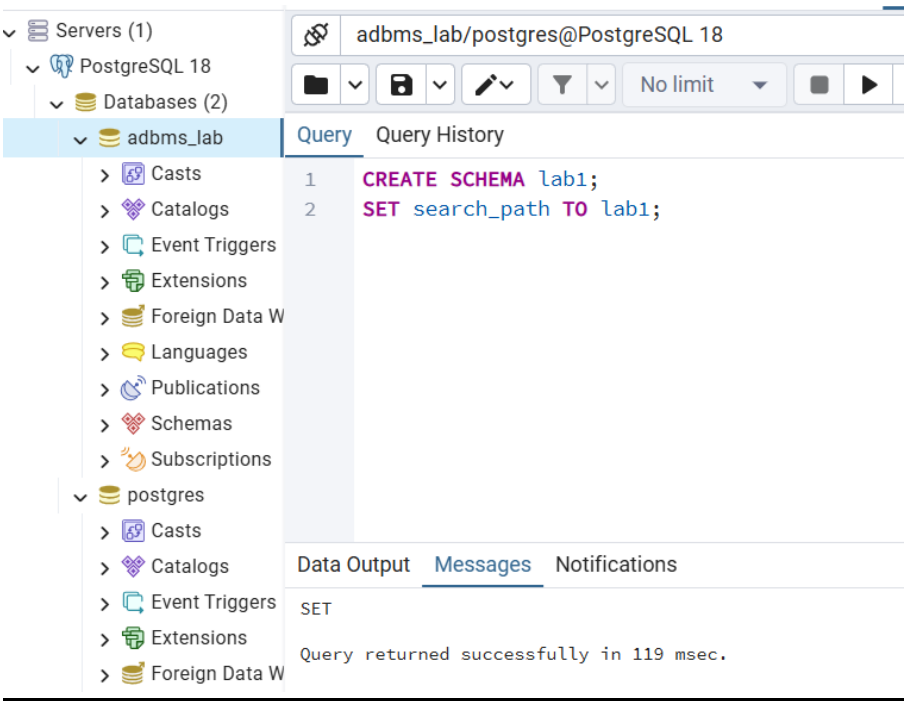
Submission Date: 10/16/2026

Lab Setup in pgAdmin

CREATE SCHEMA lab1;

SET search_path TO lab1;

Output:



Explanation:

This code creates a specific schema named lab1 to keep our tables organized. The SET search_path command tells PostgreSQL to create and look for all future tables inside this lab1 schema.

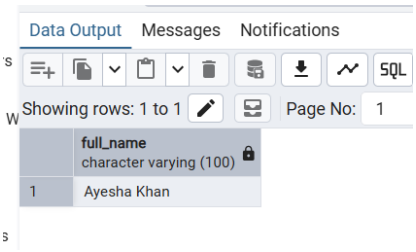
Table with an Array Column

Initial Program

Code:

```
CREATE TABLE student_profile (
   student_id INT PRIMARY KEY,
   full_name VARCHAR(100),
   phones TEXT[]
);
INSERT INTO student_profile VALUES
(1, 'Ayesha Khan', ARRAY['0300-1111111', '051-2222222']),
(2, 'Bilal Ahmed', ARRAY['0312-3333333']),
(3, 'Hina Malik', ARRAY['0321-4444444', '0333-5555555']);
SELECT * FROM student_profile;
SELECT full_name, phones[1] AS first_phone FROM student_profile;
SELECT full_name FROM student_profile WHERE '051-2222222' = ANY(phones);
```

OUTPUT:



Explanation:

The TEXT[] data type allows the phones column to store multiple values in a single row. The SELECT queries show how to retrieve the whole table, extract just the first phone number using [1], and search for a specific phone number using ANY().

Task: Add a new student and display second phone

Code:

```
INSERT INTO student_profile VALUES

(4, 'Omar Sheikh', ARRAY['0345-6666666', '051-7777777', '0300-8888888']);

SELECT full_name, phones[2] AS second_phone FROM student_profile;
```

OUTPUT:

Showing rows: 1 to 4 Page No: 1

	full_name character varying (100)	second_phone text
1	Ayesha Khan	051-2222222
2	Bilal Ahmed	[null]
3	Hina Malik	0333-5555555
4	Omar Sheikh	051-7777777

Explanation:

We inserted a new student with an array of three phone numbers. The SELECT query uses phones[2] to specifically extract the second element of the array for every row.

Object Types (Composite Types)

Initial Program

Code:

```
CREATE TYPE address_t AS (
   street VARCHAR(80),
   city VARCHAR(40),
   postal_code VARCHAR(10)
);
```

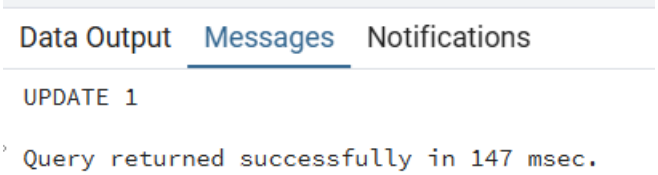
```
CREATE TABLE employee (
   emp_id INT PRIMARY KEY,
   emp_name VARCHAR(100),
   home_address address_t
);

INSERT INTO employee VALUES
(1, 'Sara Ahmed', ROW('House 12, Street 5', 'Rawalpindi', '46000')),
(2, 'Usman Ali', ROW('Flat 3, Block B', 'Islamabad', '44000'));

SELECT * FROM employee;
SELECT emp_name, (home_address).city FROM employee;

UPDATE employee SET home_address.city = 'Lahore' WHERE emp_id = 2;
```

OUTPUT:



Explanation:

We created a custom composite type address_t. The ROW() function is used to insert the multi-part address into a single column. The brackets (home_address).city are required to extract a specific part of the object.

Task: Display street and change postal code

Code:

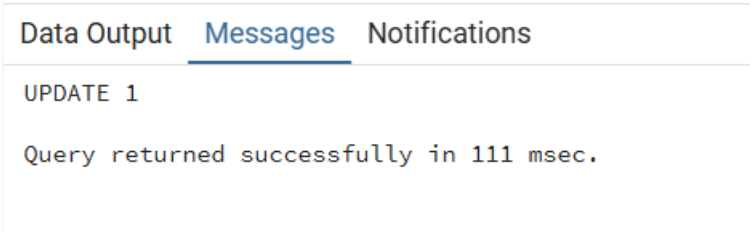
```
SELECT emp_name, (home_address).street AS street FROM employee;

UPDATE employee

SET home_address.postal_code = '47000'

WHERE emp_id = 1;
```

OUTPUT:



Explanation:

We extracted the street component of the object type using the dot notation. The UPDATE statement successfully modified a single internal part (postal code) of the composite object for Sara Ahmed.

Inheritance

Initial Program

Code:

```
CREATE TABLE person (
    person_id SERIAL PRIMARY KEY,
    name VARCHAR(100),
    email VARCHAR(100)
);


CREATE TABLE teacher (
    department VARCHAR(50)
```

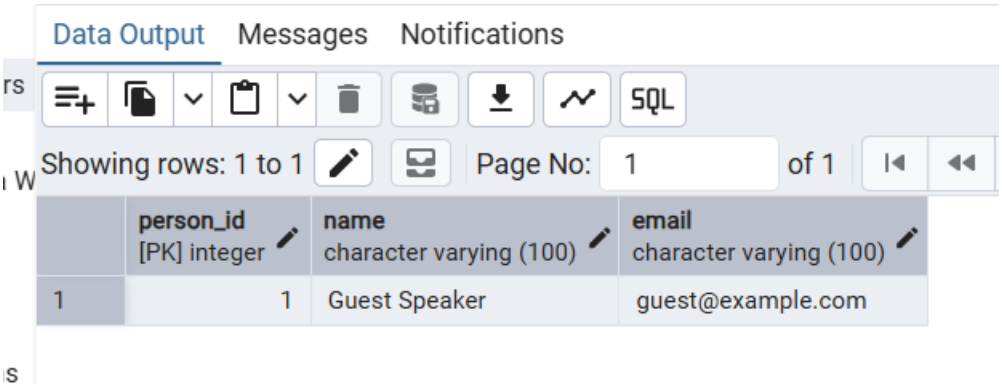
```
) INHERITS (person);


CREATE TABLE student (
   roll_no VARCHAR(15)
) INHERITS (person);


INSERT INTO person (name, email) VALUES ('Guest Speaker', 'guest@example.com');
INSERT INTO teacher (name, email, department) VALUES ('Dr. Farah', 'farah@uni.edu', 'Computer Science');
INSERT INTO student (name, email, roll_no) VALUES ('Zainab', 'zainab@uni.edu', '22IT-01');


SELECT * FROM person;
SELECT * FROM ONLY person;
```

OUTPUT:



Explanation:

The teacher and student tables inherited the columns from person. SELECT * FROM person displays all rows from the parent and the children. SELECT * FROM ONLY person displays only the rows that were inserted directly into the parent table.

Task: Add a teacher/student and display only student rows

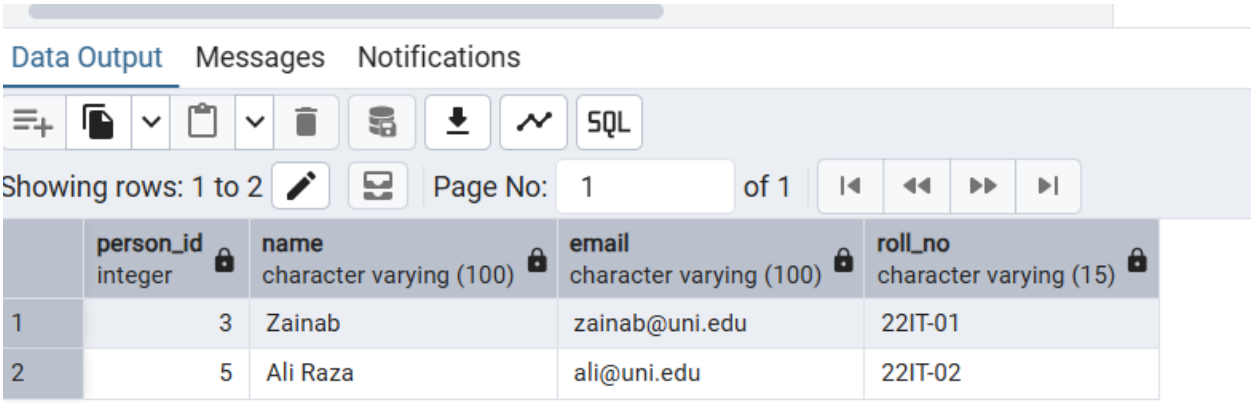
Code:

```
INSERT INTO teacher (name, email, department) VALUES ('Dr. Imran', 'imran@uni.edu', 'Software Engineering');

INSERT INTO student (name, email, roll_no) VALUES ('Ali Raza', 'ali@uni.edu', '22IT-02');


SELECT * FROM student;
```

OUTPUT:



	person_id integer	name character varying (100)	email character varying (100)	roll_no character varying (15)
1	3	Zainab	zainab@uni.edu	22IT-01
2	5	Ali Raza	ali@uni.edu	22IT-02

Explanation:

We added new data into the child tables. Querying the student table directly bypasses the parent table and only shows the rows that belong specifically to students.

7. Polymorphism

Initial Program

Code:

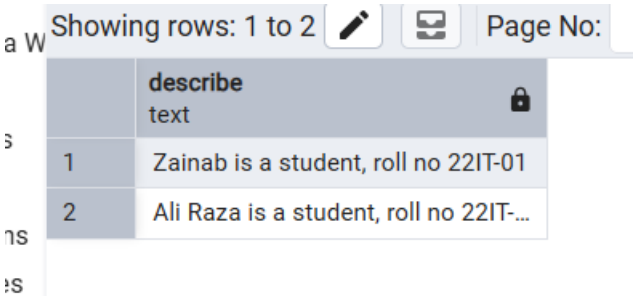
```
CREATE FUNCTION describe(t teacher) RETURNS text AS $$
   SELECT t.name || ' is a teacher of ' || t.department;
$$ LANGUAGE sql;


CREATE FUNCTION describe(s student) RETURNS text AS $$
```

```
    SELECT s.name || ' is a student, roll no ' || s.roll_no;
$$ LANGUAGE sql;


SELECT describe(t) FROM teacher t;
SELECT describe(s) FROM student s;
```

OUTPUT:


Showing rows: 1 to 2 Page No:

	describe text
1	Zainab is a student, roll no 22IT-01
2	Ali Raza is a student, roll no 22IT-...

Explanation:

We created two functions with the exact same name describe, but with different input parameters (teacher vs student). PostgreSQL automatically picks the correct function based on the table type passed to it**.**

Task: Change the sentence of each function

Code:

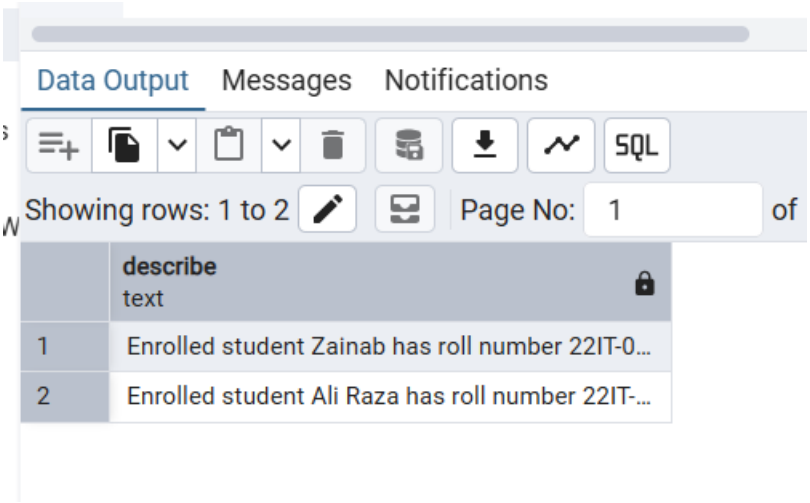
```
CREATE OR REPLACE FUNCTION describe(t teacher) RETURNS text AS $$
    SELECT 'Faculty member ' || t.name || ' belongs to the ' || t.department || ' department.';
$$ LANGUAGE sql;


CREATE OR REPLACE FUNCTION describe(s student) RETURNS text AS $$
    SELECT 'Enrolled student ' || s.name || ' has roll number ' || s.roll_no || '.';
$$ LANGUAGE sql;
```

SELECT describe(t) FROM teacher t;

SELECT describe(s) FROM student s;

OUTPUT:



Explanation:

Using CREATE OR REPLACE, we updated the logic of the functions to output different sentences while keeping the same function name, demonstrating polymorphic behavior.

Lab Questions

Q#1. Explain the difference between the relational model and the object-relational model.
Answer: The relational model stores data in simple flat tables where each cell holds a single atomic value. The object-relational model extends this by allowing complex data types (like arrays and custom objects), table inheritance, and function overloading (polymorphism), making it easier to model real-world complex data directly.

Q#2. What is an array column? Give one example where it is useful.
Answer: An array column allows a single database column to store multiple values of the same data type. It is useful for storing lists, such as multiple phone numbers for a single student, or multiple authors for a single book, avoiding the need for extra junction tables.

Q#3. How do you read one part of an object type column? Why are brackets needed?
Answer: You read a part using (column_name).part_name. The brackets are needed because, without them, PostgreSQL might confuse the dot syntax with a table/column reference. The brackets force the database to evaluate the composite object first, and then extract the specific field.

Q#4. Explain the difference between SELECT * FROM person **and** SELECT * FROM ONLY person**.**
Answer: SELECT * FROM person returns rows from the parent table as well as all child tables that inherit from it. SELECT * FROM ONLY person returns only the rows that were explicitly inserted into the parent table, ignoring the child tables.

Q#5. What is polymorphism? Explain it using the describe function.
Answer: Polymorphism means "many forms" — in databases, it allows a single function name to behave differently depending on the data type passed to it. In the lab, the describe() function prints a sentence about a teacher's department if given a teacher row, but prints a sentence about a student's roll number if given a student row.

Programming Exercises

Q#1. Execute the program and write the output

Code:

```
CREATE TABLE course (
   course_id INT PRIMARY KEY,
   title VARCHAR(50),
   topics TEXT[]
);

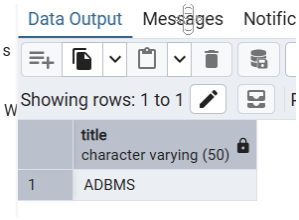
INSERT INTO course VALUES
(1, 'ADBMS', ARRAY['Joins', 'Triggers', 'Indexing']),
(2, 'Databases', ARRAY['ER Model', 'Normalization']);

SELECT title, topics[2] FROM course;
SELECT title FROM course WHERE 'Joins' = ANY(topics);
```

Output (Write this in your report):

- **First SELECT Output:**
    - ADBMS | Triggers
    - Databases | Normalization
- **Second SELECT Output:**
    - ADBMS

OUTPUT:



Q#2. Create book_catalog

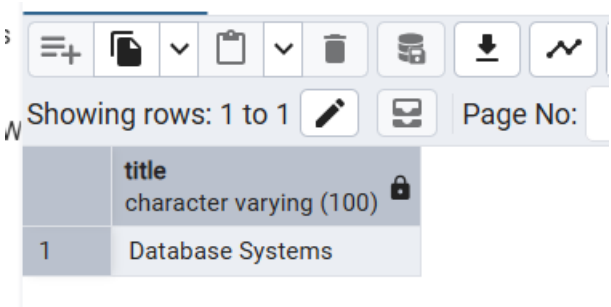
Code:

```
CREATE TABLE book_catalog (
    book_id INT PRIMARY KEY,
    title VARCHAR(100),
    authors TEXT[]
);


INSERT INTO book_catalog VALUES
(1, 'Database Systems', ARRAY['Elmasri', 'Navathe']),
(2, 'Clean Code', ARRAY['Robert Martin']),
(3, 'Intro to Algorithms', ARRAY['Cormen', 'Leiserson', 'Rivest']),
(4, 'SQL in 10 Minutes', ARRAY['Ben Forta']);


SELECT title, authors[1] AS first_author FROM book_catalog;
SELECT title FROM book_catalog WHERE 'Navathe' = ANY(authors);
```

OUTPUT:



Q#3. Create contact_t object and customer table

Code:

```
CREATE TYPE contact_t AS (
   email VARCHAR(100),
   mobile VARCHAR(20)
);

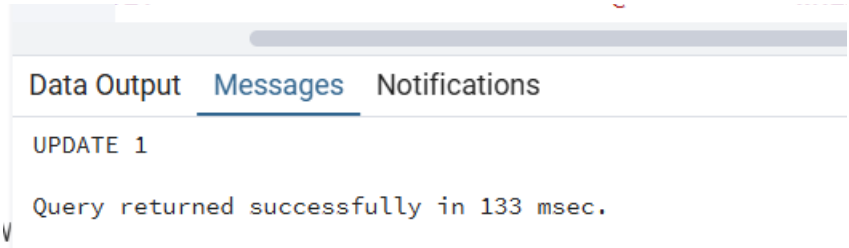
CREATE TABLE customer (
   cust_id INT PRIMARY KEY,
   cust_name VARCHAR(100),
   contact contact_t
);

INSERT INTO customer VALUES
(1, 'Ayesha Khan', ROW('ayesha@mail.com', '0300-1111111')),
(2, 'Bilal Ahmed', ROW('bilal@mail.com', '0312-3333333')),
(3, 'Hina Malik', ROW('hina@mail.com', '0321-4444444'));

SELECT cust_name, (contact).email AS email FROM customer;

UPDATE customer SET contact.email = 'bilal.new@mail.com' WHERE cust_id = 2;
```

OUTPUT:



Q#4. Parent table vehicle and child tables car & bike

Code:

```
CREATE TABLE vehicle (
   vehicle_id SERIAL PRIMARY KEY,
   brand VARCHAR(50),
   model VARCHAR(50)
);

CREATE TABLE car (
   doors INT
) INHERITS (vehicle);

CREATE TABLE bike (
   engine_cc INT
) INHERITS (vehicle);

INSERT INTO car (brand, model, doors) VALUES
('Toyota', 'Corolla', 4),
('Honda', 'Civic', 4);

INSERT INTO bike (brand, model, engine_cc) VALUES
('Yamaha', 'YBR', 125),
('Suzuki', 'GS150', 150);

INSERT INTO vehicle (brand, model) VALUES ('Generic', 'Truck-X');
```

```
SELECT * FROM vehicle;

SELECT * FROM ONLY vehicle;


CREATE FUNCTION describe_vehicle(c car) RETURNS text AS $$
   SELECT 'Car: ' || c.brand || ' ' || c.model || ' with ' || c.doors || ' doors.';
$$ LANGUAGE sql;


CREATE FUNCTION describe_vehicle(b bike) RETURNS text AS $$
   SELECT 'Bike: ' || b.brand || ' ' || b.model || ' with ' || b.engine_cc || ' cc engine.';
$$ LANGUAGE sql;


SELECT describe_vehicle(c) FROM car c;

SELECT describe_vehicle(b) FROM bike b;
```

OUTPUT:

